Step 1 — The Two-Agent Pipeline, Running Standalone (No MCP Yet) — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the Researcher, the Fact-Checker, and the revise loop between them as a plain Python program first — no MCP involved at all yet. Proving the pipeline logic works correctly before anything wraps it in a protocol is what keeps the two concerns separate. **If not:** a pipeline bug hidden behind an MCP protocol layer in Step 2 becomes two bugs to untangle instead of one.

Setup files

`requirements.txt`: `mcp[cli]`, `langchain-openai`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO` / `MAX_REVISION_ROUNDS=3` / `DEEP_RESEARCH_TIMEOUT_SECONDS=60`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    max_revision_rounds: int
    timeout_seconds: float
```

```
def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        max_revision_rounds=int(os.getenv("MAX_REVISION_ROUNDS", "3")),
        timeout_seconds=float(os.getenv("DEEP_RESEARCH_TIMEOUT_SECONDS", "60")),
    )
```

```
# src/utils/logger.py
import logging
import os
import sys

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        # stderr: the MCP server (Step 2) uses stdout as the protocol channel
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py

def load_prompt(name):
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The Researcher

```
# src/prompts/researcher.txt
Write a short, accurate 3-5 sentence summary of the topic. Be precise about
names, dates, and who did what.

{feedback_block}

Topic: {topic}
```

```
# src/agents/researcher.py
# The Researcher: draft a summary. With feedback, fix exactly the named
# issues (not a rewrite) - that's what makes round 2 different from round 1.
from dataclasses import dataclass

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.utils.prompts import load_prompt

@dataclass
class Draft:
    topic: str
    summary: str

def run_researcher(model, topic, feedback=None):
    if feedback:
        feedback_block = (
            "A fact-checker rejected your previous draft for these reasons - "
            f"fix ONLY these, keep the rest:\n{feedback}"
        )
    else:
        feedback_block = ""
    prompt = ChatPromptTemplate.from_template(load_prompt("researcher"))
    chain = prompt | model | StrOutputParser()
```

```
    summary = chain.invoke({"topic": topic, "feedback_block": feedback_block})
    return Draft(topic=topic, summary=summary)
```

The Fact-Checker

```
# src/prompts/fact_checker.txt
You fact-check a short summary. Approve ONLY if it is accurate. Reject if it
has: a wrong date, a wrong name or attribution, an invented fact, or an
overconfident claim. For each problem, write one specific issue naming the
exact error and the correction.

Topic: {topic}
Summary to check:
{summary}
```

```
# src/agents/fact_checker.py
# The Fact-Checker: a checked verdict with specific issues. Real criteria in
# the prompt, so it doesn't approve everything on round 1.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.utils.prompts import load_prompt

class FactCheckVerdict(BaseModel):
    approved: bool
    issues: list

def run_fact_checker(model, draft):
    prompt = ChatPromptTemplate.from_template(load_prompt("fact_checker"))
    chain = prompt | model.with_structured_output(FactCheckVerdict)
    return chain.invoke({"topic": draft.topic, "summary": draft.summary})
```

The pipeline

Story: this is the loop that actually ties the Researcher and Fact-Checker together, bounded by a hard `max_rounds` limit, recording what happened each round into a `Progress` holder object as it goes — that holder is what Step 4's timeout fallback later reads from. **If not:** a loop with no cap at all can run forever on a topic the Fact-Checker simply never approves (exactly Doc11's warning about unbounded revision loops).

```
# src/pipeline.py
# run_pipeline: Researcher drafts -> Fact-Checker checks -> approved? stop;
# else feed the issues back and go again, up to max_rounds. The Progress
# holder records the latest draft and each round, so the MCP layer can show a
# trace (Step 3) and fall back to the best draft on timeout (Step 4).
from dataclasses import dataclass

from src.agents.fact_checker import run_fact_checker
from src.agents.researcher import run_researcher
from src.utils.logger import get_logger

@dataclass
class PipelineResult:
    topic: str
    final_summary: str
    approved: bool
    rounds: list

class Progress:
    def __init__(self):
        self.latest_draft = None
        self.rounds = []

def run_pipeline(model, topic, max_rounds, progress=None):
    logger = get_logger(__name__)
    if progress is None:
        progress = Progress()
    draft = run_researcher(model, topic)
    for round_number in range(1, max_rounds + 1):
        progress.latest_draft = draft.summary
        verdict = run_fact_checker(model, draft)
        progress.rounds.append({
            "round": round_number,
            "approved": verdict.approved,
            "issues": verdict.issues,
        })
        logger.info(f"round {round_number}: approved={verdict.approved}")
        if verdict.approved:
            return PipelineResult(topic, draft.summary, True, progress.rounds)
        feedback = "\n".join(f"- {issue}" for issue in verdict.issues)
        draft = run_researcher(model, topic, feedback)
    progress.latest_draft = draft.summary
    return PipelineResult(topic, draft.summary, False, progress.rounds)
```

main.py (standalone)

```
# main.py  (Step 1 version)
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import get_model
from src.pipeline import run_pipeline

TOPICS = [
    "Retrieval-Augmented Generation (RAG)",
    "my opinion of the color blue",
]

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    model = get_model(config)
    for topic in TOPICS:
        result = run_pipeline(model, topic, config.max_revision_rounds)
        print(f"\n=== {topic} ===")
        for entry in result.rounds:
            mark = "rejected"
            if entry["approved"]:
                mark = "approved"
            print(f"  Round {entry['round']}: {mark}")
        print(f"approved={result.approved}\n{result.final_summary}")

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MCPRESEARCH_TEST_SETTING"
```

```
def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    if config.max_revision_rounds <= 0:
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_pipeline.py
# The revise loop and the round cap, with a tiny FAKE model (no key). The
# fake drives the Researcher (draft) and the Fact-Checker (verdict).
# Run from the repo root: python -m tests.unit.test_pipeline
from langchain_core.messages import AIMessage

from src.agents.fact_checker import FactCheckVerdict
from src.pipeline import run_pipeline

class FakeModel:
    # Researcher chain: prompt | model | StrOutputParser(); model(prompt_value)
    # must return an AIMessage. We look at whether feedback was included.
    def __call__(self, prompt_value):
```

```
        text = prompt_value.to_string()
        if "fact-checker rejected" in text:
            return AIMessage(content="RAG: Lewis et al., Meta AI, 2020.")
        return AIMessage(content="RAG was introduced by OpenAI in 2019.")

    def with_structured_output(self, schema):
        def check(prompt_value):
            text = prompt_value.to_string()
            if "OpenAI in 2019" in text:
                return FactCheckVerdict(
                    approved=False,
                    issues=["RAG is Lewis et al., Meta AI, 2020, not OpenAI."],
                )
            return FactCheckVerdict(approved=True, issues=[])
        return check

class AlwaysReject(FakeModel):
    def with_structured_output(self, schema):
        def check(prompt_value):
            return FactCheckVerdict(approved=False, issues=["nitpick"])
        return check

def test_rejects_then_approves():
    result = run_pipeline(FakeModel(), "RAG", max_rounds=3)
    ok = result.approved and len(result.rounds) == 2
    if "Meta AI" not in result.final_summary:
        ok = False
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (wrong draft -> fixed, approved r2) -> {result1}")

def test_stops_at_cap():
    result = run_pipeline(AlwaysReject(), "blue", max_rounds=2)
    ok = (not result.approved) and len(result.rounds) == 2
    result2 = "WRONG"
    if ok:
        result2 = "OK"
    print(f"case 2 (never approved -> stops at cap)    -> {result2}")

if __name__ == "__main__":
    test_rejects_then_approves()
    test_stops_at_cap()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (wrong draft -> fixed, approved r2) -> OK
case 2 (never approved -> stops at cap)    -> OK
```

Running `main.py` on a real key shows the RAG topic converging (a wrong first draft, rejected, then fixed and approved) and the "color blue" topic stopping cleanly at the cap — the loop proven before any MCP wraps it.